

14. Stored Procedures, Functions, and Triggers

14.1 Introduction

Stored Procedures, Functions, and Triggers are important database programming features in PostgreSQL. They help automate business processes, reduce repetitive code, maintain data integrity, and improve system functionality.

In the Online Book Shop Management System, these features were implemented to handle customer management, order processing, calculations, stock management, and auditing activities.

15. Stored Procedures

15.1 Stored Procedure 1 – Add New Customer

Purpose

To insert a new customer into the system.

Business Scenario

When a new customer registers on the website.

Code

```
postgres=# \c BookShopManagement
You are now connected to database "BookShopManagement" as user "postgres".
BookShopManagement=# CREATE OR REPLACE PROCEDURE add_customer(
BookShopManagement(#      p_name VARCHAR,
BookShopManagement(#      p_email VARCHAR,
BookShopManagement(#      p_phone VARCHAR,
BookShopManagement(#      p_city VARCHAR
BookShopManagement(# )
BookShopManagement=# LANGUAGE plpgsql
BookShopManagement=# AS $$
BookShopManagement$$ BEGIN
BookShopManagement$$      INSERT INTO customers(full_name, email, phone, city)
BookShopManagement$$      VALUES (p_name, p_email, p_phone, p_city);
BookShopManagement$$ END;
BookShopManagement$$ $$;
CREATE PROCEDURE
BookShopManagement=# CALL add_customer('John Doe','john@gmail.com','0771234567','Colombo');
CALL
```

Expected Result

A new customer record is inserted into the customers table.

```
BookShopManagement=# SELECT *
BookShopManagement=# FROM customers
BookShopManagement=# WHERE full_name = 'John Doe';
 customer_id | full_name | email | phone | city | created_at
-----+-----+-----+-----+-----+-----
      10001 | John Doe | john@gmail.com | 0771234567 | Colombo | 2026-06-11 21:10:44.259915
(1 row)
```

Conclusion

This procedure simplifies customer registration and centralizes insertion logic.

15.2 Stored Procedure 2 – Place Order

Purpose

To create a customer order and related order item records.

Business Scenario

A customer purchases a book from the online store.

Code

```
BookShopManagement=# CREATE OR REPLACE PROCEDURE place_order(
BookShopManagement(#     p_customer_id INT,
BookShopManagement(#     p_book_id INT,
BookShopManagement(#     p_qty INT
BookShopManagement(# )
BookShopManagement=# LANGUAGE plpgsql
BookShopManagement=# AS $$
BookShopManagement$$ DECLARE
BookShopManagement$$     book_price NUMERIC;
BookShopManagement$$ BEGIN
BookShopManagement$$     SELECT price INTO book_price
BookShopManagement$$     FROM books
BookShopManagement$$     WHERE book_id = p_book_id;
BookShopManagement$$     INSERT INTO orders(customer_id, total_amount, status)
BookShopManagement$$     VALUES (p_customer_id, book_price * p_qty, 'Pending');
BookShopManagement$$     INSERT INTO order_items(order_id, book_id, quantity, subtotal)
BookShopManagement$$     VALUES (currval(pg_get_serial_sequence('orders', 'order_id')),
BookShopManagement$$     p_book_id,
BookShopManagement$$     p_qty,
BookShopManagement$$     book_price * p_qty);
BookShopManagement$$ END;
BookShopManagement$$ $$;
CREATE PROCEDURE
```

```
BookShopManagement=# CALL place_order(1, 5, 2);  
CALL
```

A new order and order item record are created.

Conclusion

This procedure automates the order placement process.

15.3 Stored Procedure 3 – Sales Report

Purpose

To calculate sales between two dates.

Business Scenario

Management wants to view total sales for a specific period.

Code

```
BookShopManagement=# CREATE OR REPLACE PROCEDURE sales_report(  
BookShopManagement(#      start_date DATE,  
BookShopManagement(#      end_date DATE  
BookShopManagement(# )  
BookShopManagement=# LANGUAGE plpgsql  
BookShopManagement=# AS $$  
BookShopManagement$$ DECLARE  
BookShopManagement$$      total NUMERIC;  
BookShopManagement$$ BEGIN  
BookShopManagement$$  
BookShopManagement$$      SELECT SUM(total_amount)  
BookShopManagement$$      INTO total  
BookShopManagement$$      FROM orders  
BookShopManagement$$      WHERE order_date BETWEEN start_date AND end_date;  
BookShopManagement$$  
BookShopManagement$$      RAISE NOTICE 'Total Sales: %', total;  
BookShopManagement$$  
BookShopManagement$$ END;  
BookShopManagement$$ $$;  
CREATE PROCEDURE
```

Example Execution

```
CALL sales_report('2026-06-01','2026-12-11');
```

Expected Result

Total sales amount is displayed.

Conclusion

This procedure helps management analyze sales performance.

16. Functions

16.1 Function 1 – Calculate Discount Price

Purpose

Calculate the discounted price of a book.

Business Scenario

Apply a 10% discount during promotions.

Code

```
BookShopManagement=# CREATE OR REPLACE FUNCTION get_discount(price NUMERIC)
BookShopManagement=# RETURNS NUMERIC
BookShopManagement=# AS $$
BookShopManagement$$ BEGIN
BookShopManagement$$     RETURN price - (price * 0.10);
BookShopManagement$$ END;
BookShopManagement$$ $$ LANGUAGE plpgsql;
CREATE FUNCTION
```

```
BookShopManagement=# SELECT get_discount(1000);
get_discount
-----
          900.00
(1 row)
```

Conclusion

Used for promotional price calculations.

16.2 Function 2 – Count Customer Orders

Purpose

Count the number of orders placed by a customer.

Business Scenario

Show customer purchase history statistics.

Code

```
BookShopManagement=# CREATE OR REPLACE FUNCTION count_customer_orders(cid INT)
BookShopManagement=# RETURNS INT
BookShopManagement=# AS $$
BookShopManagement$$ DECLARE total INT;
BookShopManagement$$ BEGIN
BookShopManagement$$     SELECT COUNT(*) INTO total
BookShopManagement$$     FROM orders
BookShopManagement$$     WHERE customer_id = cid;
BookShopManagement$$     RETURN total;
BookShopManagement$$ END;
BookShopManagement$$ $$ LANGUAGE plpgsql;
CREATE FUNCTION
```

```
BookShopManagement=# SELECT count_customer_orders(1);
count_customer_orders
-----
                        1
(1 row)
```

Expected Result

Displays the number of orders placed by customer 1.

Conclusion

Useful for customer activity analysis.

16.3 Function 3 – Total Revenue

Purpose

Calculate revenue between two dates.

Business Scenario

Generate financial reports.

Code

```
BookShopManagement=# CREATE OR REPLACE FUNCTION total_revenue(start_date DATE, end_date DATE)
BookShopManagement=# RETURNS NUMERIC
BookShopManagement=# AS $$
BookShopManagement$$ DECLARE total NUMERIC;
BookShopManagement$$ BEGIN
BookShopManagement$$     SELECT SUM(total_amount)
BookShopManagement$$     INTO total
BookShopManagement$$     FROM orders
BookShopManagement$$     WHERE order_date BETWEEN start_date AND end_date;
BookShopManagement$$     RETURN total;
BookShopManagement$$ END;
BookShopManagement$$ $$ LANGUAGE plpgsql;
CREATE FUNCTION

BookShopManagement=# SELECT total_revenue('2026-05-29','2026-06-01');
total_revenue
-----
    39261779.42
(1 row)
```

Expected Result

Displays total revenue generated during the specified period.

Conclusion

Useful for management reporting and decision-making.

17. Triggers

17.1 Trigger 1 – Login Activity Logging

Purpose

Automatically record customer login activity.

Business Scenario

Track user login history.

Trigger Type

AFTER INSERT

Trigger Function , Trigger

```
BookShopManagement=# CREATE OR REPLACE FUNCTION log_login()
BookShopManagement=# RETURNS TRIGGER
BookShopManagement=# AS $$
BookShopManagement$$ BEGIN
BookShopManagement$$     INSERT INTO login_logs(customer_id)
BookShopManagement$$     VALUES (NEW.customer_id);
BookShopManagement$$
BookShopManagement$$     RETURN NEW;
BookShopManagement$$ END;
BookShopManagement$$ $$ LANGUAGE plpgsql;
CREATE FUNCTION
BookShopManagement=# CREATE TRIGGER trg_login
BookShopManagement=# AFTER INSERT ON customers
BookShopManagement=# FOR EACH ROW
BookShopManagement=# EXECUTE FUNCTION log_login();
CREATE TRIGGER
```

Test Execution

```

BookShopManagement=# INSERT INTO customers (
BookShopManagement(#      full_name,
BookShopManagement(#      email,
BookShopManagement(#      phone,
BookShopManagement(#      city
BookShopManagement(# )
BookShopManagement=# VALUES (
BookShopManagement(#      'Test Customer',
BookShopManagement(#      'test@gmail.com',
BookShopManagement(#      '0771234567',
BookShopManagement(#      'Colombo'
BookShopManagement(# );
INSERT 0 1
BookShopManagement=# SELECT *
BookShopManagement=# FROM login_logs
BookShopManagement=# ORDER BY log_id DESC
BookShopManagement=# LIMIT 5;
  log_id | customer_id |      login_time      |      logout_time
-----+-----+-----+-----
10001 |      10002 | 2026-06-11 22:47:52.3242 |
10000 |       8135 | 2026-05-22 17:12:21.542071 | 2026-05-22 17:13:21.542
071
  9999 |       2637 | 2026-05-22 17:13:21.542071 | 2026-05-22 17:14:21.542
071
  9998 |       4866 | 2026-05-22 17:14:21.542071 | 2026-05-22 17:15:21.542
071
  9997 |       2232 | 2026-05-22 17:15:21.542071 | 2026-05-22 17:16:21.542
071
(5 rows)

BookShopManagement=# |

```

Expected Result

A new record is inserted into the login_logs table.

Conclusion

Improves security and user activity monitoring.

17.2 Trigger 2 – Update Book Stock

Purpose

Automatically reduce stock when an order is placed.

Business Scenario

Inventory management.

Trigger Type

AFTER INSERT

Trigger Function , Trigger

```
BookShopManagement=# CREATE OR REPLACE FUNCTION reduce_stock()
BookShopManagement=# RETURNS TRIGGER
BookShopManagement=# AS $$
BookShopManagement$$ BEGIN
BookShopManagement$$
BookShopManagement$$     UPDATE books
BookShopManagement$$     SET stock = stock - NEW.quantity
BookShopManagement$$     WHERE book_id = NEW.book_id;
BookShopManagement$$
BookShopManagement$$     RETURN NEW;
BookShopManagement$$
BookShopManagement$$ END;
BookShopManagement$$ $$ LANGUAGE plpgsql;
CREATE FUNCTION
BookShopManagement=# CREATE TRIGGER trg_stock
BookShopManagement=# AFTER INSERT ON order_items
BookShopManagement=# FOR EACH ROW
BookShopManagement=# EXECUTE FUNCTION reduce_stock();
CREATE TRIGGER
```

Test Execution

```

BookShopManagement=# SELECT book_id, title, stock
BookShopManagement=# FROM books
BookShopManagement=# WHERE book_id = 1;
  book_id | title  | stock
-----+-----+-----
         1 | Book 1 |    47
(1 row)

BookShopManagement=# INSERT INTO order_items (
BookShopManagement(#      order_id,
BookShopManagement(#      book_id,
BookShopManagement(#      quantity,
BookShopManagement(#      subtotal
BookShopManagement(# )
BookShopManagement=# VALUES (
BookShopManagement(#      1,
BookShopManagement(#      1,
BookShopManagement(#      3,
BookShopManagement(#      3000
BookShopManagement(# );
INSERT 0 1
BookShopManagement=# SELECT book_id, title, stock
BookShopManagement=# FROM books
BookShopManagement=# WHERE book_id = 1;
  book_id | title  | stock
-----+-----+-----
         1 | Book 1 |    44
(1 row)

```

Expected Result

Book stock decreases automatically.

Conclusion

Ensures accurate inventory levels.

17.3 Trigger 3 – Book Audit Logging

Purpose

Record changes made to book information.

Business Scenario

Maintain audit history of price updates.

Trigger Type

AFTER UPDATE

Trigger Function , Trigger

```
BookShopManagement=# CREATE TABLE book_audit (  
BookShopManagement(#     audit_id SERIAL,  
BookShopManagement(#     book_id INT,  
BookShopManagement(#     old_price NUMERIC,  
BookShopManagement(#     new_price NUMERIC,  
BookShopManagement(#     changed_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP  
BookShopManagement(# );  
CREATE TABLE  
BookShopManagement=# CREATE OR REPLACE FUNCTION audit_book_update()  
BookShopManagement-# RETURNS TRIGGER  
BookShopManagement-# AS $$  
BookShopManagement$$ BEGIN  
BookShopManagement$$  
BookShopManagement$$     INSERT INTO book_audit(book_id, old_price, new_price)  
BookShopManagement$$     VALUES (OLD.book_id, OLD.price, NEW.price);  
BookShopManagement$$  
BookShopManagement$$     RETURN NEW;  
BookShopManagement$$  
BookShopManagement$$ END;  
BookShopManagement$$ $$ LANGUAGE plpgsql;  
CREATE FUNCTION  
BookShopManagement=# CREATE TRIGGER trg_audit  
BookShopManagement-# AFTER UPDATE ON books  
BookShopManagement-# FOR EACH ROW  
BookShopManagement-# EXECUTE FUNCTION audit_book_update();  
CREATE TRIGGER
```

Test Execution

```
BookShopManagement=# UPDATE books  
BookShopManagement-# SET price = 2500  
BookShopManagement-# WHERE book_id = 1;  
UPDATE 1  
BookShopManagement=# SELECT *  
BookShopManagement-# FROM book_audit  
BookShopManagement-# ORDER BY audit_id DESC  
BookShopManagement-# LIMIT 5;  
  audit_id | book_id | old_price | new_price |      changed_at  
-----+-----+-----+-----+-----  
          2 |        1 | 2574.83 | 2500.00 | 2026-06-11 23:03:05.868947  
          1 |        1 | 2574.83 | 2574.83 | 2026-06-11 22:49:21.263404  
(2 rows)
```

Expected Result

A new audit record is inserted into the book_audit table.

Conclusion

Provides accountability and change tracking.

18. Overall Conclusion

The implementation of Stored Procedures, Functions, and Triggers enhanced the functionality of the Online Book Shop Management System. Procedures automated business operations, functions performed reusable calculations, and triggers ensured automatic system actions. These features improved maintainability, consistency, and data integrity within the PostgreSQL database.